

DOCUMENT LINEAGE REPORT

VALID

PO_NOT_FOUND

invoice_10119.pdf

End-to-end trace of one PDF through every table in the
idp_test_dev Intelligent Document Processing pipeline

Catalog / schema	idp_test_dev.idp_test_project
Volume	raw_documents/incoming
Document ID (SHA-256)	82cab592043beee3a078eb35e3e7be0f...
Tables examined	12 of 12 pipeline tables
Report generated	2026-09-09

Executive summary

What this document is

Document type	invoice
Invoice number	INV-20250010
Purchase order ref	PO-13352
Invoice date / due date	2025-01-17 / 2025-01-17
Seller (vendor)	Riverbend Marketing Studio
Buyer (customer)	Keystone Logistics Partners
Currency	USD
Total amount	\$4,560.79

The five line items

#	Description	Qty	Unit price	Amount
1	Creative design (hours)	8	\$93.48	\$747.84
2	Ad spend management (hours)	12	\$78.47	\$941.64
3	Landing page build	1	\$1,159.52	\$1,159.52
4	Performance report	1	\$385.23	\$385.23
5	Campaign setup	1	\$1,326.56	\$1,326.56
	Subtotal			\$4,560.79

Pipeline outcome

validation_status

VALID

Reached every gold table it qualifies for.
Totals reconcile exactly (variance \$0.00).

Three findings

- Address fields mis-split

ai_parse_document put part of the Bill To address into the Ship To column.
- Invoice has no matching PO

Reconciliation returns PO_NOT_FOUND — PO-13352 is absent from gold layer.
- Local copy ≠ volume copy

sample_data/invoice_10119.pdf differs in bytes from the ingested file.

Path	.../raw_documents/incoming/
File name	invoice_10119.pdf
Extension	pdf
Size on volume	3,141 bytes
Last modified	2026-09-08 12:35:09
Pages	1
Page size	612 × 792 pt (US Letter)

Verified — matches the downloaded volume file exactly.

sample_data/invoice_10119.pdf is 3,215 bytes with SHA-256 7fae6805... — the same invoice, but different bytes, so it would hash to a new document id rather than match this row.

Notes: Please include the invoice number with your payment. For questions, contact Accounts Receivable.

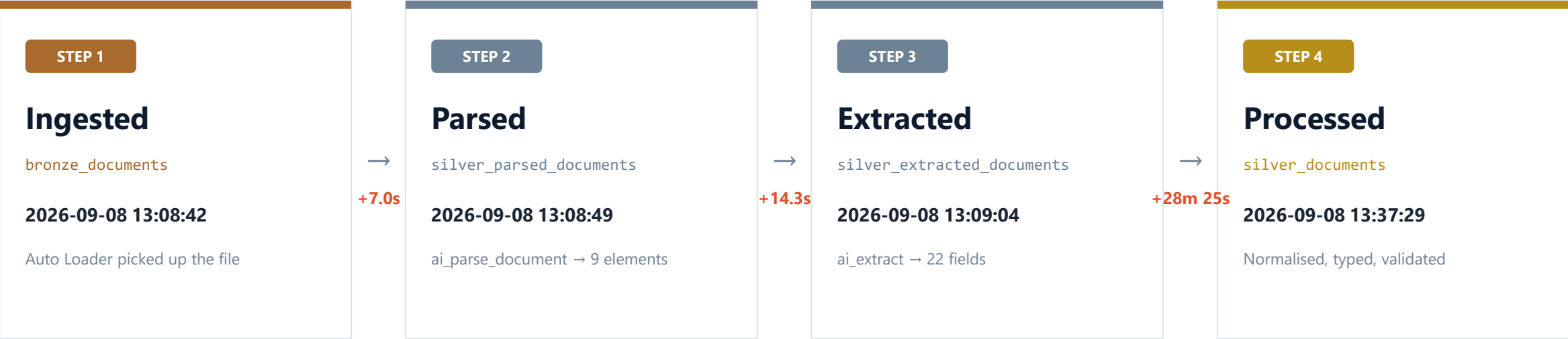
idp_test_dev.idp_test_project · invoice_10119.pdf

Where this document appears

Every table in idp_test_dev.idp_test_project was queried for this document_id. Zero-row results are by design, not gaps.

#	Table	Type	Layer	Rows for this doc	Meaning
1	bronze_documents	Streaming table	Bronze	1	Raw binary + file metadata
2	silver_parsed_documents	Streaming table	Silver	1	ai_parse_document output
3	silver_extracted_documents	Streaming table	Silver	1	ai_extract output
4	silver_documents	Streaming table	Silver	1	Normalised + validated header
5	silver_document_lines	Streaming table	Silver	5	One row per line item
6	gold_documents	Materialized view	Gold	1	Passed validation
7	gold_document_lines	Materialized view	Gold	5	Lines of valid documents
8	gold_invoices	Materialized view	Gold	1	Invoice-shaped projection
9	gold_purchase_orders	Materialized view	Gold	0	Correct — not a PO
10	gold_receipts	Materialized view	Gold	0	Correct — not a receipt
11	gold_invoice_po_reconciliation	Materialized view	Gold	1	Match attempt vs POs
12	gold_vendor_spend	Materialized view	Gold	1	Rolls into Jan-2025 vendor row

Processing timeline



Reading the gaps

Parse took +7.0s and extraction a further +14.3s — these are the two AI-function calls and they dominate cost. The +28m 25s gap before processed_at is not latency: silver_documents was refreshed in a later pipeline update, so its current_timestamp() reflects that run rather than the original arrival.

bronze_documents

Auto Loader (cloudFiles / binaryFile) reads the volume incrementally. This table stores the raw bytes plus file metadata; nothing has been interpreted yet.

document_id	82cab592043beee3a078eb35e3e7be...
source_file_path	dbfs:/Volumes/idp_test_dev/...
source_file_name	invoice_10119.pdf
file_extension	pdf
file_size_bytes	3,141

source_modified_at	2026-09-08 12:35:09
ingested_at	2026-09-08 13:08:42
document_content	BINARY, 3,141 bytes
Filter applied	extension in 10 supported types
Row count for this doc	1

Why file_size_bytes equals document_content length

Auto Loader's length column and the stored binary agree at 3,141 bytes, confirming the whole file was captured. The document_id is the SHA-256 of exactly those bytes, which makes re-ingesting an identical file idempotent at the identity level — but note it also means any byte-level change (even re-saving the PDF) produces a brand-new document_id and therefore a second row rather than an update.

silver_parsed_documents

ai_parse_document (version 2.0) turned the PDF into 9 layout elements with bounding boxes and confidence scores. Output JSON is 2,734 bytes.

id	Type	Confidence	Content (truncated)
0	text	1.0000	Riverbend Marketing ↵ Studio
1	title	1.0000	INVOICE
2	text	1.0000	22 Kingfisher Ave ↵ Nashville, TN 37203 ↵ (615) 555-0172 • invoices@riverbendstudi...
3	table	0.9854	<table><tr><td>Invoice #:</td><td>INV-20250010</td></tr><tr><td>Invoice Date:</td>...
4	table	0.8974	<table><tr><th>Bill To</th><th>Ship To</th></tr><tr><td>Keystone Logistics Partner...
5	table	0.9999	<table><tr><th>Description</th><th>Qty</th><th>Unit Price</th><th>Amount</th></tr>...
6	text	1.0000	Remit To: Riverbend Marketing Studio ↵ PO Box 1903 ↵ Nashville, TN 37202
7	table	0.9869	<table><tr><td>Subtotal</td><td>\$4,560.79</td></tr><tr><td>Tax (0%)</td><td>\$0.00<...
8	text	1.0000	Notes: Please include the invoice number with your payment. For questions, contact...

Element 4 is the weak link — confidence 0.8974, the lowest on the page

The Bill To / Ship To table was emitted as <td>Keystone Logistics Partners
3400 River Rd</td><td>Harrisburg, PA 17110</td>.

In the source PDF both cells hold the same full address and their text overlaps visually; the parser split it at the wrong boundary, leaving the city/state/ZIP stranded in the Ship To column. Everything downstream inherits this — the defect originates at parse time, not at extraction.

silver_extracted_documents

ai_extract (version 2.1) mapped the parsed layout onto the 22-field business schema. error_message is null; output JSON is 1,773 bytes. Values shown as returned, before normalisation.

Schema field	Extracted value
bill_to_address	Keystone Logistics Partners, 3400 River Rd
buyer_name	Keystone Logistics Partners
currency	USD
discount_amount	NULL
document_date	2025-01-17
document_number	INV-20250010
document_type	invoice
due_date	2025-01-17
invoice_number	INV-20250010
notes	Please include the invoice number with you...
payment_method	Wire Transfer

Schema field	Extracted value
payment_terms	Due on Receipt
purchase_order_number	PO-13352
receipt_number	NULL
requested_ship_date	NULL
seller_name	Riverbend Marketing Studio
ship_to_address	Harrisburg, PA 17110
shipping_amount	NULL
subtotal	4560.79
tax_amount	0
total_amount	4560.79

Four nulls, and they are all correct

discount_amount, shipping_amount, receipt_number and requested_ship_date are genuinely absent from this invoice, so a null is the right answer — ai_extract did not hallucinate placeholder values. tax_amount came back as 0 rather than null because the document states "Tax (0%) \$0.00" explicitly. The only questionable pair is bill_to_address / ship_to_address, and that fault was inherited from the parse stage rather than introduced here.

silver_documents

Values are flattened out of the variant, trimmed, cast to typed columns (dates, DECIMAL(18,2)) and run through the validation rules.

document_type	invoice
business_document_number	INV-20250010
document_number	INV-20250010
invoice_number	INV-20250010
purchase_order_number	PO-13352
receipt_number	NULL
document_date	2025-01-17
due_date	2025-01-17
requested_ship_date	NULL

currency	USD
seller_name	Riverbend Marketing Studio
buyer_name	Keystone Logistics Partners
bill_to_address	Keystone Logistics Partner...
ship_to_address	Harrisburg, PA 17110
payment_method	Wire Transfer
payment_terms	Due on Receipt
notes	Please include the invoice...
line_items	5 (array<variant>)

subtotal	\$4,560.79
tax_amount	\$0.00
shipping_amount	NULL
discount_amount	NULL
total_amount	\$4,560.79
calculated_total	\$4,560.79
total_variance	\$0.00
validation_status	VALID
processed_at	09-08 13:37:29

The arithmetic check passes cleanly

$\text{calculated_total} = \text{subtotal} + \text{tax} + \text{shipping} - \text{discount} = 4,560.79 + 0.00 + 0 - 0 = \$4,560.79$, which equals the stated `total_amount`, so `total_variance` is \$0.00. That sits well inside the \$0.02 tolerance, and `document_type`, `business_document_number` and `total_amount` are all present, so the validation chain falls through to `VALID`. Note the pipeline coalesces nulls to 0 in this formula, so missing shipping and discount do not break the check.

silver_document_lines

posexplode_outer over line_items produced 5 rows. Each is independently re-checked: quantity × unit_price is compared against the stated line amount.

#	Description	Qty	Unit price	Line amount	Calculated	Variance	Status
1	Creative design (hours)	8	\$93.48	\$747.84	\$747.84	\$0.00	VALID
2	Ad spend management (hours)	12	\$78.47	\$941.64	\$941.64	\$0.00	VALID
3	Landing page build	1	\$1,159.52	\$1,159.52	\$1,159.52	\$0.00	VALID
4	Performance report	1	\$385.23	\$385.23	\$385.23	\$0.00	VALID
5	Campaign setup	1	\$1,326.56	\$1,326.56	\$1,326.56	\$0.00	VALID
	Sum of line amounts			\$4,560.79			

Lines reconcile to the header

All five lines carry zero variance, and their sum — \$4,560.79 — matches the header subtotal exactly. Because tax, shipping and discount are all zero or null on this invoice, that same figure is also the document total, so header and line grain agree without any reconciling items. line_number came straight from the document (source_line_number 1-5), so the derived fallback position was not needed.

gold_documents · gold_invoices

gold_documents is silver_documents filtered to validation_status = VALID; this document passes, so it appears unchanged. gold_invoices then reshapes it with invoice-oriented column names.

gold_documents

document_type	invoice
business_document_number	INV-20250010
invoice_number	INV-20250010
purchase_order_number	PO-13352
seller_name	Riverbend Marketing Studio
buyer_name	Keystone Logistics Partners
total_amount	\$4,560.79
validation_status	VALID

gold_invoices (renamed projection)

invoice_number	INV-20250010
purchase_order_number	PO-13352
invoice_date ← document_date	2025-01-17
due_date	2025-01-17
vendor_name ← seller_name	Riverbend Marketing Studio
customer_name ← buyer_name	Keystone Logistics Partners
total_amount	\$4,560.79
payment_terms	Due on Receipt

gold_purchase_orders and gold_receipts return zero rows — correctly

Both views filter on document_type, so an invoice is excluded by design. The empty result is evidence the classifier worked, not evidence of a missing row. Of the 12 tables, these two are the only ones where this document legitimately does not appear.

gold_document_lines · gold_invoice_po_reconciliation

gold_document_lines

All 5 silver lines survive the inner join against valid documents. The variance and calculated columns are dropped for consumers.

#	Description	Qty	Unit price	Line amount	Currency	Status
1	Creative design (hours)	8	\$93.48	\$747.84	USD	VALID
2	Ad spend management (hours)	12	\$78.47	\$941.64	USD	VALID
3	Landing page build	1	\$1,159.52	\$1,159.52	USD	VALID
4	Performance report	1	\$385.23	\$385.23	USD	VALID
5	Campaign setup	1	\$1,326.56	\$1,326.56	USD	VALID

gold_invoice_po_reconciliation

Invoice side	
invoice_number	INV-20250010
invoice_po_number	PO-13352
invoice_vendor_name	Riverbend Marketing Studio
invoice_currency	USD
invoice_total	\$4,560.79

match_status

PO_NOT_FOUND

The invoice cites PO-13352, but no row in gold_purchase_orders carries that number, so the left join found nothing and every purchase_order_* column is null. This is a content gap, not a pipeline fault — the referenced PO has simply never been ingested. The join is a left join, so the invoice is still reported rather than silently dropped.

gold_vendor_spend

This document does not have its own row here — it is aggregated. It contributes entirely to the January 2025 row for its vendor.

spend_month	vendor_name	Currency	Type	Docs	Subtotal	Tax	Shipping	Total spend	Contains this doc?
2025-01	Riverbend Marketing Studio	USD	invoice	1	\$4,560.79	\$0.00	NULL	\$4,560.79	YES — this document
2025-02	Riverbend Marketing Studio	USD	invoice	1	\$2,750.00	\$220.00	\$45.00	\$2,915.00	No — different invoice

Reading the aggregate

Riverbend Marketing Studio has two monthly rows. The 2025-01 row is this invoice alone: document_count 1, subtotal \$4,560.79, tax \$0.00 and total_spend \$4,560.79 all match the header exactly. The 2025-02 row (\$2,915.00) belongs to a different Riverbend invoice and is shown only to make clear which row this document drives. shipping_amount is null on the January row because SUM over a single null input returns null rather than zero.

Grain check

date_trunc('month', document_date) on 2025-01-17 gives 2025-01-01, which is why the invoice lands in January. Since gold_vendor_spend reads only gold_documents, a document that failed validation would silently vanish from vendor spend — worth remembering when reconciling these numbers against source systems. The grain here is month × vendor × currency × document_type, so this one invoice can never appear as a row of its own.

Field-level traceability

Each business field followed from the printed PDF through to its final gold value.

Field	On the PDF	ai_extract	silver_documents	gold_invoices	OK
Invoice number	INV-20250010	INV-20250010	INV-20250010	INV-20250010	Y
PO number	PO-13352	PO-13352	PO-13352	PO-13352	Y
Invoice date	2025-01-17	2025-01-17	2025-01-17 (DATE)	2025-01-17	Y
Due date	2025-01-17	2025-01-17	2025-01-17 (DATE)	2025-01-17	Y
Terms	Due on Receipt	Due on Receipt	Due on Receipt	Due on Receipt	Y
Currency	USD	USD	USD	USD	Y
Payment method	Wire Transfer	Wire Transfer	Wire Transfer	(not projected)	Y
Seller	Riverbend Marketing Studio	Riverbend Marketing Studio	Riverbend Marketing Studio	vendor_name (same)	Y
Buyer	Keystone Logistics Partners	Keystone Logistics Partners	Keystone Logistics Partners	customer_name (same)	Y
Bill to	Keystone..., 3400 River Rd, Harrisburg PA 17110	Keystone..., 3400 River Rd	Keystone..., 3400 River Rd	(not projected)	N
Ship to	same as bill to	Harrisburg, PA 17110	Harrisburg, PA 17110	(not projected)	N
Subtotal	\$4,560.79	4560.79	4560.79	4560.79	Y
Tax	\$0.00	0	0.00	0.00	Y
Total	\$4,560.79	4560.79	4560.79	4560.79	Y

12 of 14 fields are faithful end to end. The two address fields are the single defect, and both trace back to the parse-stage table split.

Findings and recommendations

1

Bill To / Ship To are mis-split at parse time

ai_parse_document emitted the address table with the wrong cell boundary (confidence 0.8974, the page's lowest), so "Harrisburg, PA 17110" landed in Ship To while Bill To lost its city, state and ZIP. ai_extract then faithfully reproduced the bad split.

Recommendation: Consider validating that ship_to_address parses as a full address, or treat a Ship To that is shorter than its Bill To as suspect. Re-running with descriptionElementTypes enabled may also improve table segmentation.

2

The referenced purchase order is not in the lakehouse

Reconciliation returns PO_NOT_FOUND: the invoice cites PO-13352 but gold_purchase_orders holds no such number, so no three-way match is possible for this invoice.

Recommendation: This is a coverage gap in the source data, not a code defect. If PO matching matters, track the PO_NOT_FOUND rate as a pipeline health metric rather than inspecting invoices one at a time.

3

The local sample file no longer matches the ingested one

sample_data/invoice_10119.pdf is 3,215 bytes with SHA-256 7fae6805..., while the volume copy is 3,141 bytes with SHA-256 82cab592... — the value that became document_id. The invoice content is the same; the file was evidently regenerated after upload.

Recommendation: Because document_id is a content hash, re-uploading the local copy would create a second row for the same invoice rather than updating the existing one. Re-sync the local sample or drop it from the repo to avoid an accidental duplicate.

Overall: the pipeline handled this document correctly. It classified, validated and reconciled as intended, reaching all 10 tables it should and none of the 2 it should not.

How this report was produced

Every figure in this deck came from a live query against the workspace; nothing is transcribed by hand.

Workspace	dbc-6166fe71-87d7.cloud.databricks.com
Target / catalog	dev → idp_test_dev.idp_test_project
Compute	Serverless Starter Warehouse (2X-Small)
Access path	SQL Statement Execution API 2.0
Auth	OAuth U2M bearer token via Databricks CLI v1.15.0
Source PDF	Downloaded from the volume, SHA-256 verified
Tables queried	12 (all non-internal tables in the schema)
Filter	document_id = '82cab592043beee3a078eb...'

Reproducing this

The same numbers can be pulled in VS Code: open the Databricks extension's Catalog Explorer, attach the Serverless Starter Warehouse, and run the query above against any of the 12 tables. The document_id is the single join key that links every stage of the pipeline together.

Representative query

```
SELECT *
FROM idp_test_dev.idp_test_project
    .silver_documents
WHERE document_id =
    '82cab592043beee3a078eb35e...';
```

Excluded from scope

12 internal __materialization_mat_* tables and the pipeline event_log table. These are Lakeflow implementation details rather than published outputs of the medallion model.